



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA DEPARTAMENTO DE
SISTEMAS Y COMPUTACIÓN

SEMESTRE 9

Enero - Junio 2026

Ingeniería en Sistemas Computacionales

Patrones de diseño

Reporte de práctica - Patrón de diseño Composite

Diaz Morales Katherine Giselle 22210302

Maribel Guerrero Luis

Tijuana Baja California, a 19 de marzo del 2026

PATRÓN DE DISEÑO COMPOSITE

Aplicado a venta de máquinas armadas

Composite es un patrón de diseño estructural que permite componer objetos en estructuras de árbol y trabajar con esas estructuras como si fueran objetos individuales.

El uso del patrón Composite sólo tiene sentido cuando el modelo central de la aplicación puede representarse en forma de árbol.

La desventaja es que puede aumentar la complejidad del código y dificultar su depuración, mientras que sus ventajas son:

- Permite tratar a un grupo de objetos de manera uniforme, lo que facilita su manejo y organización.
- Permite construir estructuras jerárquicas y manejarlas de manera recursiva.
- Provee una interfaz común para todos los objetos del grupo, lo que facilita su extensión y modificación.

Para esta práctica se realizará un sistema de venta de computadoras el cual le permite al usuario armar su propia PC seleccionando componentes de diferentes categorías y gamas.

Código

Dentro del código tenemos componente como clase abstracta que declara la interfaz común para todos los objetos como nombre, descripción, categoría y gama.

Luego está el archivo, el cual representa los objetos hoja sin hijos (un componente individual), es decir, procesador, pantalla y disco, además no puede tener hijos por lo que AgregarHijo no hace nada.

Por último tenemos al Directorio (composite), representa una computadora armada, puede contener múltiples componentes y también calcula el precio sumando recursivamente el precio de todos sus hijos.

Componentes.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Composite_venta_de_máquinas
{
    public abstract class Componente
    {
        string _nombre;
        string _descripcion;
        string _categoria;
        string _gama;

        public Componente(string nombre, string descripcion, string categoria, string gama)
        {
            _nombre = nombre;
            _descripcion = descripcion;
            _categoria = categoria;
            _gama = gama;
        }

        public string Nombre => _nombre;
        public string Descripcion => _descripcion;
        public string Categoria => _categoria;
        public string Gama => _gama;

        public abstract void AgregarHijo(Componente c);
        public abstract IList<Componente> ObtenerHijos();
        public abstract double ObtenerPrecio { get; }
    }
}
```

Archivo.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Composite_venta_de_máquinas
{
    public class Archivo : Componente
    {
        double _precio;
        string _marca;

        public Archivo(string nombre, string descripcion, string categoria, string gama, string marca,
double precio)
            : base(nombre, descripcion, categoria, gama)
        {
            _precio = precio;
            _marca = marca;
        }

        public string Marca => _marca;
        public double Precio => _precio;

        public override void AgregarHijo(Componente c) { }

        public override IList<Componente> ObtenerHijos() => null;

        public override double ObtenerPrecio => _precio;
    }
}
```

Directorio.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Composite_venta_de_máquinas
{
    public class Directorio : Componente
    {
        private List<Componente> _hijos;

        public Directorio(string nombre, string descripcion, string categoria, string gama)
            : base(nombre, descripcion, categoria, gama)
        {
            _hijos = new List<Componente>();
        }

        public override void AgregarHijo(Componente c)
```

```

    {
        _hijos.Add(c);
    }

    public override IList<Componente> ObtenerHijos()
    {
        return _hijos.ToArray();
    }

    public override double ObtenerPrecio
    {
        get
        {
            double total = 0;
            foreach (var item in _hijos)
            {
                total += item.ObtenerPrecio;
            }
            return total;
        }
    }
}

```

Program.cs

```

using System;
using System.Collections.Generic;
using System.Linq;

namespace Composite_venta_de_máquinas
{
    class Program
    {
        static List<Archivo> catalogo = new List<Archivo>();

        static void Main(string[] args)
        {
            InicializarCatalogo();

            Console.WriteLine("===== PATRON COMPOSITE =====");

            Archivo archivo_1 = new Archivo("archivo1.txt", "Documento de texto", "Documentos", "Baja",
"Generico", 10);
            Archivo archivo_2 = new Archivo("archivo2.txt", "Documento de texto", "Documentos", "Baja",
"Generico", 20);
            Archivo archivo_3 = new Archivo("archivo3.txt", "Documento de texto", "Documentos", "Baja",
"Generico", 50);
            Archivo archivo_4 = new Archivo("archivo4.txt", "Documento de texto", "Documentos", "Baja",
"Generico", 100);
            Archivo archivo_5 = new Archivo("archivo5.txt", "Documento de texto", "Documentos", "Baja",
"Generico", 2000);

            Directorio root = new Directorio("raiz", "Directorio raiz", "Sistema", "Mixta");

```

```

Directorio dir_1 = new Directorio("dir1", "Primer directorio", "Sistema", "Mixta");
Directorio dir_2 = new Directorio("dir2", "Segundo directorio", "Sistema", "Mixta");
Directorio dir_3 = new Directorio("dir3", "Tercer directorio", "Sistema", "Mixta");

dir_1.AgregarHijo(archivo_1);
dir_2.AgregarHijo(archivo_2);
dir_3.AgregarHijo(archivo_3);
dir_3.AgregarHijo(archivo_4);
dir_1.AgregarHijo(dir_3);
root.AgregarHijo(dir_1);
root.AgregarHijo(dir_2);
root.AgregarHijo(archivo_5);

Console.WriteLine("\n--- CALCULO DE PRECIOS ---");
Console.WriteLine($"El precio del directorio {root.Nombre} es ${root.ObtenerPrecio}");
Console.WriteLine($"El precio del directorio {dir_1.Nombre} es ${dir_1.ObtenerPrecio}");
Console.WriteLine($"El precio del directorio {dir_2.Nombre} es ${dir_2.ObtenerPrecio}");
Console.WriteLine($"El precio del directorio {dir_3.Nombre} es ${dir_3.ObtenerPrecio}");

Console.WriteLine("\n--- ESTRUCTURA JERARQUICA ---");
MostrarEstructura(root, 0);

Console.ReadKey();

int opcion;
do
{
    Console.Clear();
    Console.WriteLine("===== TIENDA DE COMPUTADORAS =====");
    Console.WriteLine("1. Armar una computadora");
    Console.WriteLine("2. Ver catalogo de componentes");
    Console.WriteLine("3. Salir");
    Console.Write("Seleccione: ");

    if (int.TryParse(Console.ReadLine(), out opcion))
    {
        switch (opcion)
        {
            case 1: ArmarComputadora(); break;
            case 2: VerCatalogo(); break;
            case 3: Console.WriteLine("Gracias!"); break;
            default: Console.WriteLine("Opcion no valida"); break;
        }
    }

    if (opcion != 3)
    {
        Console.ReadKey();
    }
} while (opcion != 3);
}

```

```

static void MostrarEstructura(Componente componente, int nivel)
{
    string indent = new string(' ', nivel * 3);

    if (componente is Directorio dir)
    {
        Console.WriteLine($"{indent}+ {dir.Nombre} ({dir.ObtenerPrecio})");
        foreach (Componente hijo in dir.ObtenerHijos())
        {
            MostrarEstructura(hijo, nivel + 1);
        }
    }
    else if (componente is Archivo arch)
    {
        Console.WriteLine($"{indent}- {arch.Nombre} - ${arch.Precio}");
    }
}

static void InicializarCatalogo()
{
    catalogo.Add(new Archivo("Monitor LG 19\"", "Pantalla LED 19 pulgadas", "Pantalla", "Baja",
"LG", 120));
    catalogo.Add(new Archivo("Monitor Dell 22\"", "Pantalla Full HD 22\"", "Pantalla", "Media",
"Dell", 180));
    catalogo.Add(new Archivo("Monitor Samsung 24\"", "Pantalla IPS 24\" 75Hz", "Pantalla",
"Media", "Samsung", 220));
    catalogo.Add(new Archivo("Monitor LG 27\"", "Pantalla 4K UHD 27\"", "Pantalla", "Alta", "LG",
380));
    catalogo.Add(new Archivo("Monitor Dell 32\"", "Pantalla Curva 32\" 144Hz", "Pantalla", "Alta",
"Dell", 550));

    catalogo.Add(new Archivo("Intel Celeron", "Procesador dual-core basico", "Procesador",
"Baja", "Intel", 80));
    catalogo.Add(new Archivo("AMD Athlon", "Procesador economico", "Procesador", "Baja",
"AMD", 85));
    catalogo.Add(new Archivo("Intel Core i3", "Procesador de entrada", "Procesador", "Media",
"Intel", 150));
    catalogo.Add(new Archivo("AMD Ryzen 3", "Procesador gama media", "Procesador", "Media",
"AMD", 160));
    catalogo.Add(new Archivo("Intel Core i5", "Procesador rendimiento", "Procesador", "Alta",
"Intel", 250));
    catalogo.Add(new Archivo("AMD Ryzen 5", "Procesador alto rendimiento", "Procesador",
"Alta", "AMD", 260));
    catalogo.Add(new Archivo("Intel Core i7", "Procesador tope gama", "Procesador", "Alta",
"Intel", 380));
    catalogo.Add(new Archivo("AMD Ryzen 7", "Procesador tope gama", "Procesador", "Alta",
"AMD", 390));

    catalogo.Add(new Archivo("HDD 500GB", "Disco duro mecanico", "Almacenamiento", "Baja",
"Seagate", 35));

```

```

        catalogo.Add(new Archivo("SSD 240GB", "Unidad solida basica", "Almacenamiento", "Baja",
"Kingston", 30));
        catalogo.Add(new Archivo("SSD 480GB", "SSD capacidad media", "Almacenamiento",
"Media", "Kingston", 55));
        catalogo.Add(new Archivo("SSD 1TB", "SSD alta capacidad", "Almacenamiento", "Alta",
"Samsung", 120));
        catalogo.Add(new Archivo("SSD NVMe 500GB", "SSD ultra rapido", "Almacenamiento", "Alta",
"WD", 90));
        catalogo.Add(new Archivo("SSD NVMe 1TB", "SSD tope gama", "Almacenamiento", "Alta",
"Samsung", 180));
    }

```

```

static void VerCatalogo()
{
    Console.Clear();
    Console.WriteLine("===== CATALOGO POR CATEGORIAS =====");

    var categorias = catalogo.GroupBy(c => c.Categoria);

    foreach (var categoria in categorias)
    {
        Console.WriteLine($"\\n--- {categoria.Key.ToUpper()} ---");

        var porGama = categoria.GroupBy(c => c.Gama);

        foreach (var gama in porGama)
        {
            Console.WriteLine($" [{gama.Key}]");
            foreach (var comp in gama)
            {
                Console.WriteLine($" {comp.Nombre} - {comp.Marca} - ${comp.Precio:F2}");
                Console.WriteLine($" {comp.Descripcion}");
            }
        }
    }
}

```

```

static void ArmarComputadora()
{
    Console.Clear();
    Console.WriteLine("===== ARMAR TU COMPUTADORA =====");

    Console.WriteLine("\\nElige la GAMA que deseas para tu PC:");
    Console.WriteLine("1. Baja (economica)");
    Console.WriteLine("2. Media (balanceada)");
    Console.WriteLine("3. Alta (rendimiento)");
    Console.Write("Opcion: ");

    string gamaElegida = "";
    int opcGama = int.Parse(Console.ReadLine());
    switch (opcGama)
    {

```



```

        case 1: gamaElegida = "Baja"; break;
        case 2: gamaElegida = "Media"; break;
        case 3: gamaElegida = "Alta"; break;
        default: gamaElegida = "Media"; break;
    }

    Directorio miPC = new Directorio("Mi PC", "Computadora personalizada", "PC Completa",
gamaElegida);

    Console.WriteLine($"Armando PC GAMA {gamaElegida.ToUpper()}");

    Console.WriteLine("\n--- CATEGORIA: PANTALLA ---");
    var pantallas = catalogo.Where(c => c.Categoria == "Pantalla" && c.Gama ==
gamaElegida).ToList();

    if (pantallas.Count == 0)
    {
        Console.WriteLine($"No hay pantallas de gama {gamaElegida}. Mostrando todas:");
        pantallas = catalogo.Where(c => c.Categoria == "Pantalla").ToList();
    }

    var pantallaElegida = SeleccionarComponente(pantallas, "pantalla");
    if (pantallaElegida != null)
    {
        miPC.AgregarHijo(pantallaElegida);

        if (pantallaElegida.Marca == "LG")
            Console.WriteLine("\nLG ofrece buena relacion calidad-precio en pantallas.");
        else if (pantallaElegida.Marca == "Dell")
            Console.WriteLine("\nDell es conocido por su excelente reproduccion de color.");
    }

    Console.WriteLine("\n--- CATEGORIA: PROCESADOR ---");
    var procesadores = catalogo.Where(c => c.Categoria == "Procesador" && c.Gama ==
gamaElegida).ToList();

    if (procesadores.Count == 0)
        procesadores = catalogo.Where(c => c.Categoria == "Procesador").ToList();

    var procesadorElegido = SeleccionarComponente(procesadores, "procesador");
    if (procesadorElegido != null) miPC.AgregarHijo(procesadorElegido);

    Console.WriteLine("\n--- CATEGORIA: ALMACENAMIENTO ---");

    Console.Write("Que capacidad deseas? (240, 480, 500, 1000): ");
    string cap = Console.ReadLine();

    var almacenamiento = catalogo.Where(c => c.Categoria == "Almacenamiento" &&
        c.Nombre.Contains(cap) &&
        c.Gama == gamaElegida).ToList();

    if (almacenamiento.Count == 0)

```

```
almacenamiento = catalogo.Where(c => c.Categoria == "Almacenamiento" && c.Gama == gamaElegida).ToList();
```

```
if (almacenamiento.Count == 0)
    almacenamiento = catalogo.Where(c => c.Categoria == "Almacenamiento").ToList();
```

```
var almacenElegido = SeleccionarComponente(almacenamiento, "almacenamiento");
if (almacenElegido != null) miPC.AgregarHijo(almacenElegido);
```

```
MostrarResumen(miPC);
```

```
Console.WriteLine("\n===== SUGERENCIAS =====");
Console.WriteLine($"Para una PC de gama {gamaElegida}:");
```

```
if (gamaElegida == "Baja")
    Console.WriteLine(" * Considera actualizar el almacenamiento a SSD en el futuro.");
else if (gamaElegida == "Media")
    Console.WriteLine(" * Esta configuracion es ideal para oficina y multimedia.");
else if (gamaElegida == "Alta")
    Console.WriteLine(" * Excelente para diseño, edicion de video y gaming.");
}
```

```
static Archivo SeleccionarComponente(List<Archivo> componentes, string tipo)
{
    Console.WriteLine($"Seleccione {tipo}:");
    for (int i = 0; i < componentes.Count; i++)
    {
        Console.WriteLine($"{i + 1}. {componentes[i].Nombre} - {componentes[i].Marca} -
${componentes[i].Precio:F2} [{componentes[i].Gama}]");
        Console.WriteLine($" {componentes[i].Descripcion}");
    }
    Console.WriteLine("0. Omitir");
    Console.Write("Opcion: ");

    int opc = int.Parse(Console.ReadLine());
    if (opc > 0 && opc <= componentes.Count)
        return componentes[opc - 1];

    return null;
}
```

```
static void MostrarResumen(Directorio pc)
{
    Console.Clear();
    Console.WriteLine("\n===== PC ARMADA =====");
    Console.WriteLine($"Gama: {pc.Gama}");
    Console.WriteLine($"Precio total: ${pc.ObtenerPrecio:F2}");
    Console.WriteLine("\nComponentes:");

    foreach (Componente hijo in pc.ObtenerHijos())
    {
        if (hijo is Archivo a)
```

```
{
    Console.WriteLine($" * {a.Nombre} ({a.Marca}) - {a.Categoria} [{a.Gama}] -  

    ${a.Precio:F2}");
    Console.WriteLine($"    {a.Descripcion}");
}
}
}
}
```

Captura

Primero tenemos el menú

```
===== TIENDA DE COMPUTADORAS =====
1. Armar una computadora
2. Ver catalogo de componentes
3. Salir
Seleccione:
```

El catálogo tiene 3 categorías:

```

===== CATALOGO POR CATEGORIAS =====

--- PANTALLA ---
[Baja]
Monitor LG 19" - LG - $120.00
Pantalla LED 19 pulgadas

[Media]
Monitor Dell 22" - Dell - $180.00
Pantalla Full HD 22"
Monitor Samsung 24" - Samsung - $220.00
Pantalla IPS 24" 75Hz

[Alta]
Monitor LG 27" - LG - $380.00
Pantalla 4K UHD 27"
Monitor Dell 32" - Dell - $550.00
Pantalla Curva 32" 144Hz

--- PROCESADOR ---
[Baja]
Intel Celeron - Intel - $80.00
Procesador dual-core basico
AMD Athlon - AMD - $85.00
Procesador economico

[Media]
Intel Core i3 - Intel - $150.00
Procesador de entrada
AMD Ryzen 3 - AMD - $160.00
Procesador gama media

[Alta]
Intel Core i5 - Intel - $250.00
Procesador rendimiento
AMD Ryzen 5 - AMD - $260.00
Procesador alto rendimiento
Intel Core i7 - Intel - $380.00
Procesador tope gama
AMD Ryzen 7 - AMD - $390.00
Procesador tope gama

```

```

--- ALMACENAMIENTO ---
[Baja]
  HDD 500GB - Seagate - $35.00
    Disco duro mecanico
  SSD 240GB - Kingston - $30.00
    Unidad solida basica
[Media]
  SSD 480GB - Kingston - $55.00
    SSD capacidad media
[Alta]
  SSD 1TB - Samsung - $120.00
    SSD alta capacidad
  SSD NVMe 500GB - WD - $90.00
    SSD ultra rapido
  SSD NVMe 1TB - Samsung - $180.00
    SSD tope gama

```

Para armar tu pc se muestra:

```
===== ARMAR TU COMPUTADORA =====

Elige la GAMA que deseas para tu PC:
1. Baja (economica)
2. Media (balanceada)
3. Alta (rendimiento)
Opcion: 3

Armando PC GAMA ALTA

--- CATEGORIA: PANTALLA ---

Seleccione pantalla:
1. Monitor LG 27" - LG - $380.00 [Alta]
   Pantalla 4K UHD 27"
2. Monitor Dell 32" - Dell - $550.00 [Alta]
   Pantalla Curva 32" 144Hz
0. Omitir
Opcion: 2

Dell es conocido por su excelente reproduccion de color.

--- CATEGORIA: PROCESADOR ---

Seleccione procesador:
1. Intel Core i5 - Intel - $250.00 [Alta]
   Procesador rendimiento
2. AMD Ryzen 5 - AMD - $260.00 [Alta]
   Procesador alto rendimiento
3. Intel Core i7 - Intel - $380.00 [Alta]
   Procesador tope gama
4. AMD Ryzen 7 - AMD - $390.00 [Alta]
   Procesador tope gama
0. Omitir
Opcion: 4

--- CATEGORIA: ALMACENAMIENTO ---
Que capacidad deseas? (240, 480, 500, 1000): 1000_
```

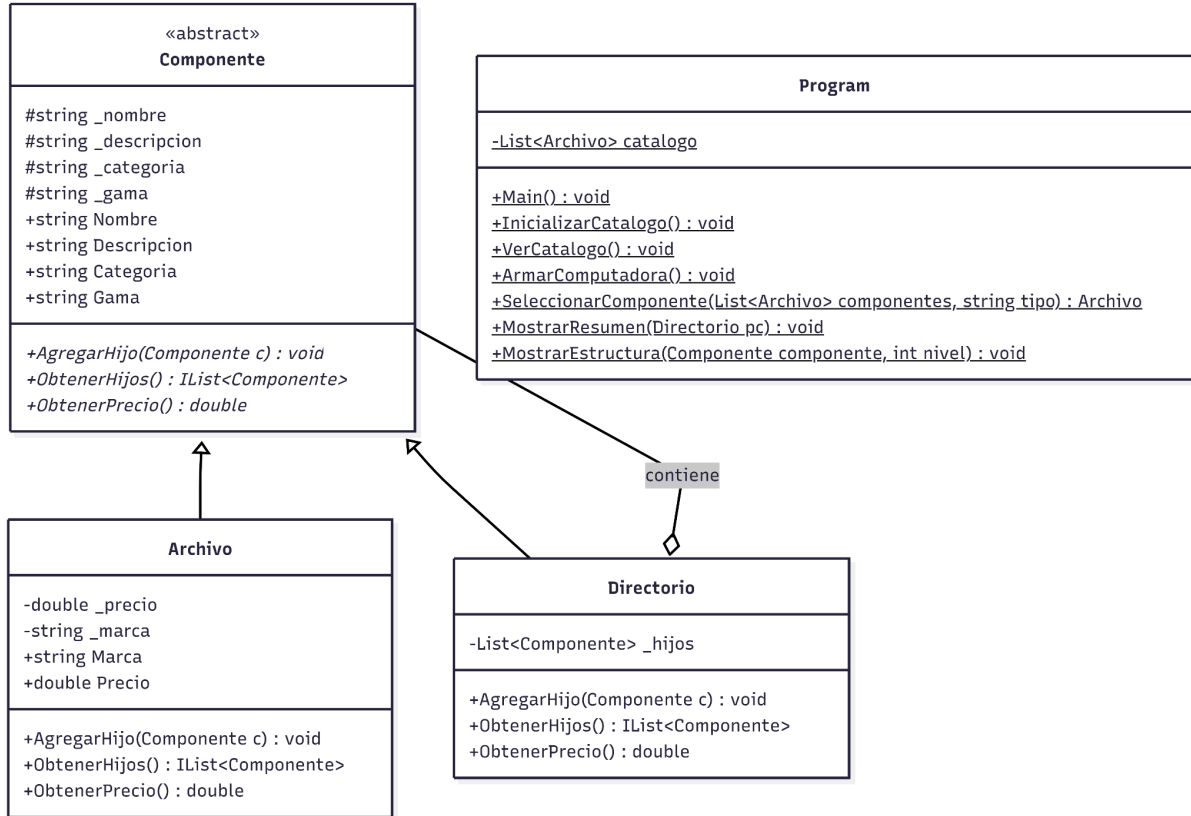
Resultado final:

```
===== PC ARMADA =====
Gama: Alta
Precio total: $1120.00

Componentes:
* Monitor Dell 32" (Dell) - Pantalla [Alta] - $550.00
  Pantalla Curva 32" 144Hz
* AMD Ryzen 7 (AMD) - Procesador [Alta] - $390.00
  Procesador tope gama
* SSD NVMe 1TB (Samsung) - Almacenamiento [Alta] - $180.00
  SSD tope gama

===== SUGERENCIAS =====
Para una PC de gama Alta:
* Excelente para diseño, edicion de video y gaming.
```

Diagrama UML



Conclusión

Dentro de la práctica se hizo uso del patrón Composite, el cual me ayudó a organizar el código de forma más lógica. También comprendí mejor el funcionamiento de este método y me gustó que para saber el precio total, solo se manda a llamar al método y solito se encarga de sumar todo, aunque tenga muchos componentes dentro. Al final el programa es práctico y funciona bien para este tipo de problemas, además es fácil de modificar.

Referencias

- Refactoring.Guru. (s.f.). Composite.
<https://refactoring.guru/es/design-patterns/composite>
- DevExpert. (s.f.). Composite: patrones de diseño.
<https://devexpert.io/blog/composite-patrones-diseno>